

Object Oriented Concepts || P1

Rahul Guha || SME || Java FS Training || HCL

Java - What is OOP?

OOP stands for Object-Oriented Programming.

Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods.

OOP is ideal for building scalable, reusable, and maintainable code.

Object-oriented programming has several advantages over procedural programming:

- OOP provides a clear structure for the programs
- OOP helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

What is Programming Paradigm ?

A programming paradigm is a way or style of programming based on certain principles and techniques.

It defines how code is written, structured, and executed. Some main programming paradigms are:

Paradigm	Description	Languages
Procedural	Step-by-step execution using functions and procedures.	C, Pascal, Fortran
Object-Oriented (OOP)	Uses objects and classes to structure code for reusability.	Java, C++, Python, C#
Functional	Focuses on pure functions, immutability, and avoiding state changes.	Haskell, Lisp, Scala, JavaScript (functional features)
Declarative	Describes what to do rather than how to do it.	SQL, Prolog, HTML

6 main pillars of OOP's are :-

- **Class** - A blueprint for creating objects (e.g., Car, Person).
- **Object** - An instance of a class (e.g., myCar, john).
- **Inheritance** - Allowing a class to inherit properties from another class.
- **Polymorphism** - Having multiple methods with the same name but different behavior (method overloading/overriding).
- **Abstraction** - Hiding complex details and showing only the essential features.
- **Encapsulation** - Hiding data using private variables and public methods.

Smalltalk, Ruby, Scala etc are purely object oriented languages.

Encapsulation

Bundle data + behavior; hide internals

Inheritance

Reuse & extend existing classes

Polymorphism

One interface, many behaviors

Abstraction

Expose essentials, hide complexity

Java is **strongly** Object Oriented Language but **not purely** Object oriented because java as primitive data types and static concept which is not related or tied to objects

What is Class in Java ?

- Class is a blueprint or prototype or template for creating objects in Java.
- It is not a real-world entity, means it does not exist physically; it only defines how an object should behave.
- Class does not occupy memory (but objects occupies memory).
- NOTE : Class metadata (such as the number of variables, variable names, method names, constructors, etc.) is stored in the Method Area.

- **Syntax**

access-modifiers class ClassName

- {
- Fields (Instance Variables) – Store object data.
- Constructors – Initialize objects.
- Methods – Define object behavior.
- Static Members – Shared among all objects.
- Nested Classes – Class inside another class.
- Blocks – Static and instance blocks for initialization.
- }

Types of Java Classes

Class Name	Definition	Where It Is Mostly Used?	Usage Frequency
Concrete Class	A regular class that can be instantiated and contains complete implementations of methods.	General Java applications and OOP-based programs.	Mostly Used
Abstract Class	A class that cannot be instantiated and may have abstract (unimplemented) methods.	Used in inheritance and polymorphism for defining base behaviors.	Mostly Used
Final Class	A class that cannot be extended (inherited) by any other class.	Used for security and immutability (e.g., String class in Java).	Mostly Used
Inner Class (Nested Class)	A class defined inside another class for better encapsulation.	Used when a class is closely associated with another class.	Not Much Used
Anonymous Inner Class	A class without a name, used for quick implementation.	Common in event handling, multi-threading, and callbacks.	Mostly Used
Static Class (Nested Static Class)	A static inner class that does not require an instance of the outer class.	Used for utility or helper methods inside outer classes.	Not Much Used
POJO Class	A simple Java class with fields, getters, setters, and no business logic.	Used in enterprise applications, data models, and APIs.	Mostly Used
Bean Class	A special POJO with a default constructor, private fields, and public getters/setters.	Used in Java frameworks like Spring and JSP.	Mostly Used
Singleton Class	A class that allows only one instance throughout the program.	Used in design patterns, logging, database connections, and configuration management.	Mostly Used
Utility Class	A class that contains only static methods and is not meant to be instantiated.	Used for helper functions and utility operations.	Mostly Used
Service Class	A class that contains business logic, commonly used in frameworks like Spring.	Used in enterprise applications and service layers.	Mostly Used
Entity Class	A class that represents a database table, used in ORM frameworks like Hibernate.	Used in database interactions and ORM (e.g., Hibernate, JPA).	Mostly Used
Record Class (Java 14+)	A special class type designed for immutable data storage with minimal code.	Used in modern applications where immutable data structures are needed.	Gaining Popularity

Method in Java

- A method in Java is a block of code that performs a specific task and can be reused multiple times.

Syntax :

- access-modifiers return-type methodName(List of Parameters)
- {
- // body
- }

Example :

- public void run(String name)
- {
- System.out.println(name+" is running");
- }

What is Object in Java ?

- Object is an instance of a class - Objects are created from a class blueprint and represent real data.
- Object is a real-world entity - It represents tangible things like a car, person, or book in programming.
- Object occupies memory - When an object is created (new ClassName()), it gets memory allocated in the heap.

Objects has its own :

- Identity - Unique memory reference assigned by JVM
- State / Attribute - Data stored in instance variables
- Behaviour - Actions (methods)

Syntax :

- 1. Creating Objects:
- `ClassName objectName = new ClassName();`
ClassName objectName; — Declares a reference variable objectName of type ClassName.
- **new ClassName();** — Creates a new object of ClassName by allocating memory and calling its constructor.
- **=** — Assigns the reference (address) of the newly created object to the variable objectName.
- **So overall:** objectName now holds a reference to a new ClassName object in memory.

Constructor in Java

- What is Constructor in Java ?
- A constructor in Java is a special type of method used to initialize objects.
- It is called when an instance of the class is created or say, when an object of a class is created.
- Whenever an object is created using the new keyword, at least one constructor is called.
Syntax :
- access-modifiers ClassName(list of parameters)
- {
- //body
- }

Example :

```
public class Test
{
    //constructor
    public Test()
    {
        //body
    }
    public static void main(String[] args)
    {
        Test t = new Test(); //constructor called
                                automatically when we create object
    }
}
```

Use of Constructor in Java

Use of Constructor:

- Constructors are used to initialize the object. It means that constructor assigns the values to the instance variables when an object is created. This can be done manually by the programmer or automatically by Java using a default constructor.
- Note : Constructors are not used to create an object, they are only used to initialize an object.

Rules for Constructors in Java

Below are rules for constructors in Java:

- The constructor must have the same name as that of class name.
- A constructor does not have a return type (not even void).
- Constructors can have any access modifier (public, private, protected or default) but cannot have non-access modifiers like static, final, abstract etc.
- The constructor is called automatically when an object is created.

Use of Constructor in Java

Types of Constructors in Java

There are 3 types of constructors in Java:-

Default Constructor (created by compiler)

No-Args (No Argument) Constructor (created by programmer)

Parameterized Constructor (created by programmer)

1. Default Constructor:

If programmer doesn't create any constructor, then Java compiler automatically creates one constructor which is known as default constructor.

It initializes an object properties with default values (0, null, false).

Default Constructor

Example :

```
public class Test
```

```
{
```

```
    //here programmer didn't created any constructor, so Java compiler will create  
    default constructor automatically.
```

```
    public static void main(String[] args)
```

```
{
```

```
        Test t = new Test();    //default constructor called automatically when we create  
        object
```

```
}
```

```
}
```

No-Args (No Argument) Constructor

If a programmer creates the constructor without any parameters, then it is known as **No Argument Constructor**.

It is used to initialize objects with default values or perform some basic setup.

Syntax :

```
class ClassName
{
    // No-Argument Constructor
    ClassName()
    {
        // Initialization code (if needed)
    }
}
```

Parameterized Constructor

If a programmer creates the constructor with parameters, then it is known as **Parameterized Constructor**.

It is used to:

Initialize objects with specific values instead of default values.

Avoid the need for setter methods after object creation.

Improve code readability by directly assigning values through the constructor.

class **ClassName**

```
{  
    ClassName(dataType parameter1, dataType parameter2)  
    {  
        // Initialization code  
    }  
}
```

Private Constructor

A private constructor cannot be accessed from outside the class. It is commonly used in:

Singleton Pattern: To ensure only one instance of a class is created.

- **Utility/Helper Classes:** To prevent instantiation of a class containing only static methods.

Why use a Private Constructor?

- Private constructors are commonly used when you **do not want other classes to directly create objects.**
- Common applications include:
- **Singleton Design Pattern**
- Utility classes
- Factory methods
- Controlling object creation

```
class Student {  
  
    private Student() {  
        System.out.println("Private Constructor Called");  
    }  
  
    public static void createObject() {  
        Student obj = new Student();  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        // Student obj = new Student(); // ✗ Error  
  
        Student.createObject(); // ✓ Allowed  
    }  
}
```